

ขั้นตอนที่ 6.4 ตัวอย่างการ Query ข้อมูลจากฐานข้อมูล

1. หาจำนวนผู้ใช้แต่ละช่วงอายุในระบบ (users) และจำนวนผู้ใช้ในแต่ละช่วงอายุที่สั่งซื้อสินค้าอย่างน้อย 1 คำสั่งซื้อขึ้นไป (customers) โดยแบ่งช่วงอายุ (age_range) ดังนี้ 16-25 ปี, 26-35 ปี, 36-46 ปี และ 47 ปีขึ้นไป

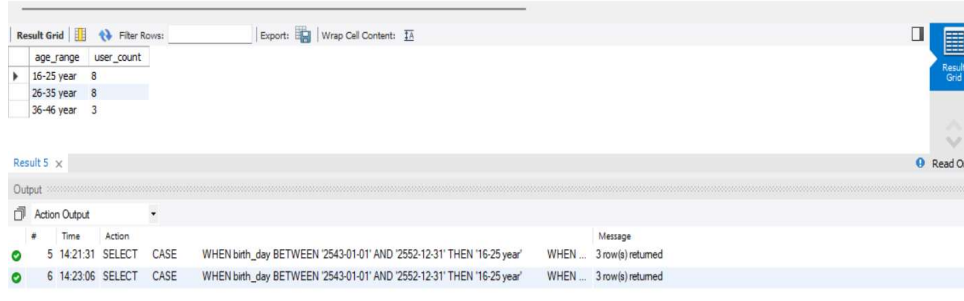
เหตุผล เพื่อใช้ในการวิเคราะห์ว่ากลุ่มลูกค้าที่จะสมัครบัญชีเพื่อใช้ในการเลือกซื้อของกับระบบเป็นกลุ่มคนอายุเท่าใด และวิเคราะห์ต่อได้ว่ากลุ่มลูกค้าที่เป็นเป้าหมายแท้จริงซึ่งเคยสั่งซื้อสินค้าจากผู้ขายในระบบเป็นกลุ่มคนอายุเท่าใด เพื่อนำสถิตินี้มาปรับใช้ในการจัดโปรโมชั่น และปรับแนวทางการขายหรือโปรโมชั่นสินค้าให้ตรงกับกลุ่มเป้าหมายมากขึ้น

SQL Query และผลลัพธ์

```

1 • SELECT
2     CASE
3         WHEN birth_day BETWEEN '2543-01-01' AND '2552-12-31' THEN '16-25 year'
4         WHEN birth_day BETWEEN '2533-01-01' AND '2542-12-31' THEN '26-35 year'
5         WHEN birth_day BETWEEN '2522-01-01' AND '2532-12-31' THEN '36-46 year'
6         WHEN birth_day <= '2521-12-31' THEN '47 year ++'
7     END AS age_range,
8     COUNT(DISTINCT users.username) AS user_count -- once for each user
9 FROM Person_id
10 JOIN users ON users.person_id_number = Person_id.person_id_number
11 JOIN orders ON users.username = orders.username -- check if user has ordered once
12 GROUP BY age_range
13 ORDER BY age_range;

```



age_range	user_count
16-25 year	8
26-35 year	8
36-46 year	3

คำอธิบาย

- ทำการ SELECT birth_day จากตาราง Person_id และทำการแบ่งตามช่วงอายุด้วย case เพื่อจะได้คอลัมป์เพิ่มขึ้นมาชื่อ age_range และทำการ JOIN Users เพื่อเอาเพียงข้อมูลของผู้ใช้ในระบบเท่านั้นโดยตั้งเงื่อนไขว่า users.person_id_number = person_id.person_id_number
- เมื่อได้ข้อมูลของผู้ใช้ทุกคนแล้วก็จะทำการ JOIN Orders เพื่อแยกเอาเฉพาะผู้ใช้ที่เคยซื้อสินค้าอย่างน้อย 1 คำสั่งซื้อ โดยตั้งเงื่อนไขว่า users.username = orders.username
- เมื่อได้ข้อมูลของผู้ใช้ที่เคยซื้อสินค้าอย่างน้อย 1 ครั้งและบอกช่วงอายุของผู้ใช้นั้น ๆ ก็จะมีการนับจำนวนผู้ใช้ในช่วงอายุนั้น ๆ โดย COUNT (DISTINCT users.username) และ GROUP BY age_range เพื่อทำการนับจำนวนผู้ใช้ในช่วงอายุโดยแบ่งกลุ่มด้วย age_range และเหตุผลที่ใช้

DISTINCT เพราะตอน JOIN Orders ผู้ใช้ 1 คนอาจเคยสั่งซื้อสินค้ามากกว่า 1 คำสั่งซื้อ และอาจทำให้แถวที่ได้มามีมากกว่า 1 แถวทั้งที่เป็นคนเดียวกัน ดังนั้นใส่ DISTINCT เพื่อนับผู้ใช้ 1 คนแค่ 1 ครั้งเท่านั้น และสุดท้าย ORDER BY age_range เพื่อแสดงผลช่วงอายุเรียงจากน้อยไปมาก

2. แสดงปริมาณคำสั่งซื้อในแต่ละช่วงเวลาของเดือนมกราคม 2025 โดยแบ่งช่วงเวลาเป็น 6 ช่วง ดังนี้
00:00-04:00 น., 04:01-08:00 น., 08:01-12:00 น., 12:01-16:00 น., 16:01-20:00 และ 20:01-00:00 น.

เหตุผล เมื่อรู้ว่าช่วงเวลาใดมีคำสั่งซื้อเยอะ แสดงว่าช่วงเวลาดังกล่าวมีผู้คนสะดวกเข้ามาดูของภายในร้านค้าเยอะ จะสามารถนำสถิติดังกล่าวมาวางแผนการขายเพื่อให้มียอดคำสั่งซื้อเพิ่มขึ้น หรือวางแผนว่าควรลงโปรโมทหรือขายสินค้าในช่วงใด

SQL Query และผลลัพธ์

```

1 • SELECT
2 CASE
3     WHEN order_time BETWEEN '00:00:00' AND '04:00:00' THEN '00:00 - 04:00'
4     WHEN order_time BETWEEN '04:00:01' AND '08:00:00' THEN '04:01 - 08:00'
5     WHEN order_time BETWEEN '08:00:01' AND '12:00:00' THEN '08:01 - 12:00'
6     WHEN order_time BETWEEN '12:00:01' AND '16:00:00' THEN '12:01 - 16:00'
7     WHEN order_time BETWEEN '16:00:01' AND '20:00:00' THEN '16:01 - 20:00'
8     WHEN order_time BETWEEN '20:00:01' AND '23:59:59' THEN '20:01 - 23:59'
9 END AS time_range ,
10 COUNT(*) AS order_count
11 FROM orders
12 WHERE order_date BETWEEN '2025-01-01' AND '2025-01-31'
13 GROUP BY time_range
14 ORDER BY time_range;

```

time_range	order_count
00:00 - 04:00	2
04:01 - 08:00	1
08:01 - 12:00	6
12:01 - 16:00	1
16:01 - 20:00	1
20:01 - 23:59	2

Result 1 Result 2 **Result 3** × Result 4 Result 5 Result 6 Result 7 Result 8 Result 9

Output

Action Output

#	Time	Action	Message
162	12:19:31	select product_id , case WHEN order_date BETWEEN '2024-01-01' AND '2024-0...	49 row(s) returned
163	12:19:31	select product_id,gender , count(gender) as จำนวนคน from users JOIN person_id ...	36 row(s) returned

คำอธิบาย

- SELECT เวลาที่สั่งซื้อสินค้า (order_time) ของแต่ละคำสั่งซื้อสินค้าจากราย Order และแบ่งช่วงให้กับข้อมูลที่ได้มาเป็น 6 ช่วงเวลาเพื่อบอกว่าคำสั่งซื้อสินค้านั้น ๆ สั่งซื้อในช่วงเวลาไหนจะได้คอลัมป์ใหม่ชื่อ time_range
- ข้อมูลที่ได้จะได้ทุกคำสั่งซื้อสินค้า แต่เราต้องการแค่ในช่วงเดือนมกราคมปี 2025 จึงใส่เงื่อนไข WHERE order_date BETWEEN '2025-01-01' AND '2025-01-31' เพื่อจะเอาเฉพาะคำสั่งซื้อสินค้าที่สั่งซื้อภายในเดือนมกราคมปี 2025 เท่านั้น จากนั้นจะทำการนับ record ของแต่ละช่วงเวลาด้วย COUNT(*) ซึ่งจะแบ่งตาม time_range จึง GROUP BY time_range จากนั้นเมื่อได้จำนวน record ของแต่ละช่วงเวลาก็จะแสดงผล ORDER BY time_range

3. ค้นหาว่ามีบัตรเอทีเอ็มของธนาคารใดบ้างที่ลูกค้าใช้ในการชำระค่าสินค้าในปี 2024 จนถึง 31/01/2025 และบัตรเอทีเอ็มแต่ละธนาคารถูกใช้ในการชำระไปทั้งหมดกี่ครั้ง

เหตุผล หากทราบว่าบัตรเอทีเอ็มของธนาคารใดที่ลูกค้าใช้เป็นส่วนใหญ่ ผู้ขายสามารถนำสถิติดังกล่าวมาพิจารณาในการจัดโปรโมชั่นส่วนลดต่างๆร่วมกันกับธนาคาร เพื่อดึงดูดและกระตุ้นให้ลูกค้าอยากซื้อสินค้ามากขึ้น

SQL Query และผลลัพธ์

```
1 • SELECT bank , COUNT(*) AS bank_count FROM payments
2 WHERE payment_type = 'บัตร ATM'
3 AND payment_date BETWEEN '2024-01-01' AND '2025-01-31'
4 GROUP BY bank
5 ORDER BY bank_count DESC;
```

Result Grid		Filter Rows:	Export:	Wrap Cell Contents	IA
bank	bank_count				
กสิกรไทย	13				
กรุงไทย	12				
ไทยพาณิชย์	3				
กรุงไทย	1				
กรุงศรี	1				

Result 14 x			
Output			
Action Output			
#	Time	Action	Message
11	14:46:07	SELECT bank FROM payments WHERE payment_type = 'บัตร ATM' AND payment_date BETWEEN '2024-01-01' A...	30 row(s) returned
12	14:46:24	SELECT bank , COUNT(*) AS bank_count FROM payments WHERE payment_type = 'บัตร ATM' AND payment_date ...	5 row(s) returned
13	14:46:42	SELECT bank , COUNT(*) AS bank_count FROM payments WHERE payment_type = 'บัตร ATM' AND payment_date ...	5 row(s) returned
14	14:48:15	SELECT bank , COUNT(*) AS bank_count FROM payments WHERE payment_type = 'บัตร ATM' AND payment_date ...	5 row(s) returned
15	14:48:18	SELECT bank , COUNT(*) AS bank_count FROM payments WHERE payment_type = 'บัตร ATM' AND payment_date ...	5 row(s) returned
16	14:48:18	SELECT bank , COUNT(*) AS bank_count FROM payments WHERE payment_type = 'บัตร ATM' AND payment_date ...	5 row(s) returned
17	14:48:19	SELECT bank , COUNT(*) AS bank_count FROM payments WHERE payment_type = 'บัตร ATM' AND payment_date ...	5 row(s) returned

คำอธิบาย

- SELECT bank from Payments เพื่อดึงข้อมูลจากการชำระเงินว่าชำระผ่านธนาคารใดบ้าง โดยตั้งเงื่อนไขว่าต้องมาจากการชำระเงินที่ชำระผ่านบัตร ATM เท่านั้น
- WHERE payment_type = 'บัตร ATM' และอยู่ในช่วงเดือนมกราคม 2024 จนถึงเดือนมกราคมปี 2025 ด้วย AND payment_date BETWEEN '2024-01-01' AND '2025-01-31'
- จากนั้นทำการนับจำนวนแถวการชำระเงินโดยแบ่งตามธนาคาร COUNT(*) GROUP BY bank จะได้จำนวนการชำระเงินที่เกิดขึ้นในช่วงวันดังกล่าวของแต่ละธนาคารและทำการเรียงจากน้อยไปมากตามจำนวนการชำระเงินด้วย ORDER BY bank_count DESC

4. ค้นหายอดขายและจำนวนชิ้นที่ขายได้ของสินค้าที่จัดโปรโมชั่นและในช่วงที่จัดโปรโมชันนั้นร้านค้าขายสินค้าได้เท่าไร

เหตุผล เมื่อทราบถึงยอดขายของสินค้าในช่วงที่จัดโปรโมชั่น ร้านค้าสามารถนำสถิติมาประกอบการพิจารณาได้ว่าโปรโมชั่นที่จัดนั้นกระตุ้นยอดขายได้จริงหรือไม่ และควรจัดหรือปรับปรุงโมชันต่อไปได้อย่างไร

SQL Query และผลลัพธ์

```

1  SELECT promotion_name,
2     promotions.product_id,
3     sellers.seller_name AS store,
4     start_date,
5     end_date,
6     sum(quantity) AS total_quantity ,
7     sum(order_details.total_price) AS total_price
8  FROM promotions
9  JOIN sellers ON SUBSTRING_INDEX(promotions.product_id, '-', -1) = sellers.seller_id
10 LEFT JOIN orders ON orders.order_date BETWEEN start_date AND end_date
11 LEFT JOIN order_details ON orders.order_id = order_details.order_id
12 AND order_details.product_id = promotions.product_id
13 GROUP BY promotions.promotion_name, promotions.product_id , sellers.seller_name, start_date , end_date
14 ;

```

promotion_name	product_id	store	start_date	end_date	total_quantity	total_price
ลดสัปดาห์เดือน	1_4	อยุธยา	2024-01-25	2024-01-30	NULL	NULL
แฟลชเซลส์แรง	1_3	ไทยรุ่งเรือง	2024-02-02	2024-02-05	NULL	NULL
ลดกระเปาะ	1_1	สมพร	2024-03-01	2024-03-31	18	432.00
ช้อปสิ้นคืนเงิน	1_5	รุ่งเรืองพาณิชย์	2024-04-09	2024-04-15	NULL	NULL
ลดสุดคุ้มวันนี้	1_4	อยุธยา	2024-03-03	2024-04-04	NULL	NULL
ช้อปคุ้มทุกวัน	3_5	รุ่งเรืองพาณิชย์	2024-04-12	2024-04-18	NULL	NULL
ลดลดหนัก	2_2	เจริญทรัพย์	2024-05-25	2024-05-30	NULL	NULL
ดีลพิเศษวันหยุด	2_4	อยุธยา	2024-06-06	2024-06-20	NULL	NULL
เชลล์พาไป	2_4	อยุธยา	2024-06-25	2024-06-30	NULL	NULL
คุ้มค่าที่สุด	1_2	เจริญทรัพย์	2024-08-07	2024-08-12	NULL	NULL
ลดใหญ่ไปกระพริบ	3_2	เจริญทรัพย์	2024-09-08	2024-09-30	4	191000.00
โปรช้อปไม่ยั้ง	1_3	ไทยรุ่งเรือง	2024-10-09	2024-10-25	NULL	NULL
ช้อปคืนไม่อั้น	2_1	สมพร	2024-12-12	2025-01-31	7	189.00
ช่วยเหลือใช้	3_4	อยุธยา	2025-01-01	2025-03-01	25	384.80
ปีใหม่มองคล	4_1	สมพร	2025-01-01	2025-01-15	35	543.20
คลายร้อน	3_4	อยุธยา	2025-01-05	2025-01-20	15	228.00
ฤดูแห่ง	4_5	รุ่งเรืองพาณิชย์	2025-01-15	2025-02-15	3	1656.00
ฉลองตรุษจีน	5_5	รุ่งเรืองพาณิชย์	2025-01-22	2025-01-30	7	800.10
Back to school	5_1	สมพร	2025-05-10	2025-05-31	NULL	NULL
Back to school	6_1	สมพร	2025-05-10	2025-05-31	NULL	NULL

คำอธิบาย

- SELECT ชื่อโปรโมชั่น, รหัสสินค้า, วัน/เดือน/ปีเริ่มโปรโมชั่น และสิ้นสุดโปรโมชั่นจากตารางโปรโมชั่น เพื่อเลือกทุกโปรโมชั่นที่เคยจัดมา และเลือกชื่อผู้ขายจากตารางผู้ขาย เพื่อแสดงว่าโปรโมชั่นดังกล่าว เป็นของผู้ขายใด
- JOIN ตาราง Sellers (ผู้ขาย) โดยมีเงื่อนไข คือ รหัสผู้ขาย (seller_id) ในตารางผู้ขาย และเลขหลังสัญลักษณ์ '_' ของรหัสสินค้า (product_id) ใช้ SUBSTRING_INDEX (promotions.product_id, '_', -1) (ซึ่ง -1 คือเอาค่าหลังเครื่องหมาย '_') ในตาราง Promotions ต้องเป็นรหัสเดียวกัน เพื่อจะตรวจสอบได้ว่าโปรโมชั่นของสินค้านั้นเป็นโปรโมชั่นของผู้ขายใด
- จากนั้น LEFT JOIN Orders เพื่อเลือกคำสั่งซื้อที่อยู่ในช่วงโปรโมชั่นจาก ON orders.order_date BETWEEN start_date and end_date โดยหากโปรโมชั่นไหนที่ไม่มีคำสั่งซื้อก็ยังถูกนำมาประมวลผลโดย LEFT JOIN
- จากนั้นก็จะทำการตรวจสอบว่าคำสั่งซื้อที่อยู่ในช่วงโปรโมชันนั้นมีการซื้อสินค้าที่ถูกจัดโปรโมชั่นในช่วงนั้นหรือไม่ โดยใช้ LEFT JOIN order_details โดยที่ order_details.order_id = orders.order_id นั่นคือเลือกรายละเอียดคำสั่งซื้อของคำสั่งซื้อนั้น ๆ และต้องเลือกที่ซื้อสินค้าตรงกับที่จัดโปรโมชั่น AND order_details.product_id = promotions.product_id โดยหากมีการซื้อสินค้าที่จัดโปรโมชั่นจะแสดงจำนวนขึ้น (quantity) และราคารวมทั้งหมด (total_price)
- เมื่อได้ข้อมูลของแต่ละโปรโมชั่นว่าถูกซื้อไปกี่ชิ้นและคิดเป็นเงินกี่บาทของแต่ละคำสั่งซื้อก็จะทำการรวมจำนวนขึ้นและจำนวนเงินทั้งหมดของโปรโมชันนั้น ๆ โดยใช้ sum (quantity) , sum (total_price)
- GROUP BY promotions.promotion_name, promotions.product_id, sellers.seller_name, start_date, end_date เพื่อจัดค่ากลุ่มข้อมูลตามชื่อโปรโมชั่น, รหัสสินค้า, ร้านค้า, วัน/เดือน/ปีเริ่มโปรโมชั่น และสิ้นสุดโปรโมชั่น

5. สินค้าขายดีในแต่ละจังหวัดในปี2024

เหตุผล เพื่อนำมาวิเคราะห์ว่าจังหวัดใด ๆ ทำไมนิยมซื้อสินค้าอันนั้น

SQL Query และผลลัพธ์

```
1 • select province, order_details.product_id , sum(quantity) as quantity_sell from address
2 JOIN delivery ON delivery.address_id = address.address_id
3 JOIN payments ON delivery.payment_id = payments.payment_id
4 JOIN order_details ON payments.order_id = order_details.order_id
5 WHERE payments.payment_date BETWEEN '2024-01-01' AND '2024-12-31'
6 GROUP BY province,product_id
7 ORDER BY province , sum(quantity) DESC;
```

province	product_id	quantity_sell
กรุงเทพมหานคร	1_1	15
กรุงเทพมหานคร	2_3	12
กรุงเทพมหานคร	3_4	10
กรุงเทพมหานคร	1_4	10
กรุงเทพมหานคร	2_1	6
กรุงเทพมหานคร	1_3	5
กรุงเทพมหานคร	5_1	5
กรุงเทพมหานคร	6_1	5
กรุงเทพมหานคร	1_5	3
กรุงเทพมหานคร	2_2	2
กรุงเทพมหานคร	3_2	2
กรุงเทพมหานคร	1_2	1
กรุงเทพมหานคร	3_5	1
กรุงเทพมหานคร	2_5	1
คลองเตย	1_4	20
ชลบุรี	1_2	1
ชลบุรี	1_5	1

province	product_id	quantity_sell
ตาก	1_4	4
ตาก	2_5	3
ตาก	4_2	3
ตาก	5_2	3
ตาก	2_4	1
นนทบุรี	3_1	5
นนทบุรี	1_4	5
นนทบุรี	1_5	3
นนทบุรี	2_5	1
บางเขน	3_2	2
ปทุมธานี	1_3	10
พิษณุโลก	1_1	8
พิษณุโลก	1_3	4
มุกดาหาร	1_4	5
มุกดาหาร	2_4	2
ระยอง	1_2	1
ลาดกระบัง	3_5	2
ลาดกระบัง	3_2	1
ส่าทูน	1_1	2

คำอธิบาย

- เลือกจังหวัดจากตาราง address จากนั้นจะทำการเลือกกว่ามีการจัดส่งใดบ้างที่ส่งไปที่อยู่นั้น ๆ โดยใช้ JOIN Delivery ON delivery.address_id = address.address_id จะได้การจัดส่งที่ถูกจัดส่งไปที่ที่อยู่แต่ละไอทีในตาราง Address
- ต่อมา JOIN Payments ON delivery.payment_id = payments.payment_id เพื่อทำการดึงข้อมูลการชำระเงินที่ถูกจัดส่งไปเพื่อใช้ตรวจสอบสินค้าที่ถูกซื้อ
- จากนั้น JOIN Order_details ON payments.order_id = order_details.order_id เพื่อดูรายละเอียดการสั่งซื้อที่อยู่ในการชำระเงินว่าซื้อสินค้าใดไปบ้างโดยแสดง product_id, จำนวน (quantity) และใช้ WHERE payments.payment_date BETWEEN '2024-01-01' AND '2024-12-31' เพื่อเอาแค่ในปี 2024 เท่านั้น ก็จะได้สินค้าที่ถูกซื้อในจังหวัดนั้น ๆ ก็ขึ้นในแต่ละออเดอร์

- สุดท้ายทำการนับจำนวนสินค้าทั้งหมดที่ถูกซื้อในจังหวัดนั้น ๆ โดยใช้ sum (quantity) as quantity_sell GROUP BY province,product_id นั่นคือจัดกลุ่มตามจังหวัดและรหัสสินค้าเพื่อนับจำนวนและให้เรียงจากน้อยไปมากเพื่อแสดงผล ORDER BY province, sum (quantity) DESC;

6. แสดงชื่อลูกค้า, รหัสคำสั่งซื้อ, วันที่ชำระเงิน, หมายเลขการจัดส่งสินค้า และสถานะการจัดส่งของสินค้าที่มีสถานะการจัดส่งสินค้ายังไม่เรียบร้อยดี โดยเรียงตามวันที่ที่สั่งซื้อจากน้อยไปมาก

เหตุผล เพื่อใช้ตรวจสอบการสั่งซื้อสินค้าที่ตกหล่นที่ยังไม่ถูกจัดส่ง

SQL Query และผลลัพธ์

```
1 • select first_name , orders.order_id , payment_date , delivery_id , delivery.delivery_status from person_id
2 JOIN users on person_id.person_id_number = users.person_id_number
3 JOIN orders on orders.username = users.username
4 JOIN payments on payments.order_id = orders.order_id
5 JOIN delivery on delivery.payment_id = payments.payment_id
6 WHERE delivery.delivery_status != 'จัดส่งสำเร็จ'
7 order by payment_date
8 ;
```

first_name	order_id	payment_date	delivery_id	delivery_status
อลงกา	48	2025-01-13	48	กำลังจัดส่ง
อภิษฐา	50	2025-01-20	50	กำลังจัดส่ง
วริศรา	49	2025-01-20	49	กำลังจัดส่ง
กรรณิ	51	2025-01-26	51	แพ็คสินค้า
อลงกา	52	2025-02-02	52	แพ็คสินค้า
วริศรา	53	2025-02-02	53	แพ็คสินค้า

#	Time	Action	Message
45	16:16:45	select first_name , orders.order_id , payment_date , delivery_id , delivery.delivery_status from users JOIN person_id on person_id...	6 row(s) returned
46	16:18:48	select first_name , orders.order_id , payment_date , delivery_id , delivery.delivery_status from person_id JOIN users on person_id...	6 row(s) returned

คำอธิบาย

- เลือกชื่อจริงของผู้ใช้จากตาราง person_id ด้วย JOIN Users ON person_id.person_id_number = users.person_id_number โดยใช้เลขบัตรประชาชนในการอ้างอิง
- จากนั้นจะทำการดูคำสั่งซื้อของผู้ใช้ทั้งหมดด้วย JOIN Orders on orders.username = users.username
- เมื่อได้คำสั่งซื้อแล้วก็ต้องเลือกเอาเฉพาะคำสั่งซื้อที่ชำระเงินแล้วในการตรวจสอบว่าถูกจัดส่งหรือยัง ด้วย JOIN Payments on payments.order_id = orders.order_id เมื่อได้คำสั่งซื้อที่ชำระเงินแล้วก็จะถูกจัดส่งในตาราง Delivery
- จากนั้นก็สามารถตรวจสอบสถานะของรายการนั้นได้โดยเราจะเลือกรายการที่ยังไม่ถูกจัดส่งสำเร็จโดยใช้ JOIN Delivery on delivery.payment_id = payments.payment_id WHERE

delivery.delivery_status != 'จัดส่งสำเร็จ' จากนั้นก็จะเรียงตามวันที่ชำระเงินเพื่อดูว่าคำสั่งซื้อสินค้าไหนที่ยังไม่ถูกจัดส่งนานที่สุด order by payment_date;

7. แสดงรหัส, ชื่อสินค้า, รหัสสต็อก และจำนวนคงเหลือของสินค้าที่มีปริมาณในสต็อกสินค้าปัจจุบันน้อยกว่า 20 ชิ้น โดยกรณีที่เป็นสินค้ารหัสเดียวกัน สินค้าที่ถูกรับเข้ามาในสต็อกจะถูกนำมาแสดงก่อน

เหตุผล เพื่อจะได้ตรวจสอบสินค้าคงเหลือในสต็อกที่รับเข้ามาก่อนได้เพื่อนำไปจำหน่ายและคำนวณต้นทุน

SQL Query และผลลัพธ์

```
1 • select products.product_id , product_name , stock_id , items_left , stocks.stock_receipt_date from products
2 JOIN stocks on products.product_id = stocks.product_id
3 WHERE items_left < 20
4 order by stocks.product_id , stocks.stock_receipt_date
5 ;
```

product_id	product_name	stock_id	items_left	stock_receipt_date
1_2	หูฟังบลูทูธ Xiaomi รุ่น AirPods	4	5	2024-02-14
1_3	นาฬิกาข้อมืออัจฉริยะ แบรนด์ 55 g.x12	1	0	2024-01-20
1_3	นาฬิกาข้อมืออัจฉริยะ แบรนด์ 55 g.x12	7	0	2024-04-13
1_3	นาฬิกาข้อมืออัจฉริยะ แบรนด์ 55 g.x12	16	10	2024-11-05
1_4	กระดาษชำระ ซอสมะเขือเทศ 200 แผ่น	2	0	2024-01-22
1_4	กระดาษชำระ ซอสมะเขือเทศ 200 แผ่น	10	0	2024-06-13
1_5	ข้าวสาร 5 กก.	6	13	2024-04-09
2_2	น้ำยาล้างจาน จีโน่ 500 ml.	9	17	2024-06-12
2_4	แอปเปิ้ล แมงกอสต์ สีดำ 60 g.	3	5	2024-02-05
2_5	สมูทตี้ A5 กระดาษหยาบ 100 แผ่น	11	15	2024-07-03
3_2	iPhone 16	13	10	2024-09-12
4_1	Fanta สีส้ม 120 mL.	20	15	2024-12-05
5_5	พัดลมพกพา รุ่น H20P 2.5 inch	25	12	2025-02-10

#	Time	Action	Message
50	16:28:47	select stocks product_id , product_name , stock_id , items_left , stocks.stock_receipt_date from products JOIN stocks on product...	13 row(s) returned
51	16:31:30	select products product_id , product_name , stock_id , items_left , stocks.stock_receipt_date from products JOIN stocks on prod...	13 row(s) returned

คำอธิบาย

- เลือกรหัสสินค้าและชื่อจากตาราง Products
- จากนั้นเลือกสต็อกสินค้าที่เป็นของสินค้าชนิดใด ๆ ทั้งหมดจากตาราง Stocks โดยใช้ JOIN Stocks on products.product_id = stocks.product_id
- จากนั้นทำการตรวจสอบสต็อกที่มีสินค้าเหลือน้อยกว่า 20 ชิ้น ด้วย WHERE items_left < 20 และแสดงผลเรียงตามรหัสสินค้าโดยที่หากสินค้าใดมีมากกว่า 1 สต็อก จะเรียงจากวันที่เข้ามาก่อน order by stocks.product_id , stocks.stock_receipt_date

8. แสดงเดือนที่มียอดขายมากที่สุดถึงเดือนที่มียอดขายน้อยที่สุดของสินค้าแต่ละรายการในปี 2024

เหตุผล เพื่อนำมาวิเคราะห์ว่าทำไมสินค้านั้นใด ๆ ถึงขายดีในช่วงเดือนนั้น

SQL Query และผลลัพธ์

```

1 • select product_id ,
2   case
3     WHEN order_date BETWEEN '2024-01-01' AND '2024-01-31' THEN 'มกราคม'
4     WHEN order_date BETWEEN '2024-02-01' AND '2024-02-29' THEN 'กุมภาพันธ์'
5     WHEN order_date BETWEEN '2024-03-01' AND '2024-03-31' THEN 'มีนาคม'
6     WHEN order_date BETWEEN '2024-04-01' AND '2024-04-30' THEN 'เมษายน'
7     WHEN order_date BETWEEN '2024-05-01' AND '2024-05-31' THEN 'พฤษภาคม'
8     WHEN order_date BETWEEN '2024-06-01' AND '2024-06-30' THEN 'มิถุนายน'
9     WHEN order_date BETWEEN '2024-07-01' AND '2024-07-31' THEN 'กรกฎาคม'
10    WHEN order_date BETWEEN '2024-08-01' AND '2024-08-31' THEN 'สิงหาคม'
11    WHEN order_date BETWEEN '2024-09-01' AND '2024-09-30' THEN 'กันยายน'
12    WHEN order_date BETWEEN '2024-10-01' AND '2024-10-31' THEN 'ตุลาคม'
13    WHEN order_date BETWEEN '2024-11-01' AND '2024-11-30' THEN 'พฤศจิกายน'
14    WHEN order_date BETWEEN '2024-12-01' AND '2024-12-31' THEN 'ธันวาคม'
15  end as month_th
16  , sum(quantity) as จำนวนชิ้นที่ขายในเดือนนั้น
17  from orders
18  JOIN order_details ON orders.order_id = order_details.order_id
19  WHERE order_date BETWEEN '2024-01-01' AND '2024-12-31'
20  group by product_id,month_th
21  order by product_id , sum(quantity) desc ;

```

product_id	month_th	จำนวนชิ้นที่ขายในเดือนนั้น
1_1	มกราคม	18
1_1	เมษายน	3
1_1	มิถุนายน	2
1_1	พฤษภาคม	2
1_2	กันยายน	1
1_2	กุมภาพันธ์	1
1_2	กรกฎาคม	1
1_3	พฤศจิกายน	40
1_3	มกราคม	16
1_3	เมษายน	5
1_3	กุมภาพันธ์	3
1_3	ธันวาคม	2
1_4	มกราคม	24
1_4	พฤศจิกายน	20
1_5	มิถุนายน	2
1_5	กรกฎาคม	2
1_5	มกราคม	1
1_5	สิงหาคม	1
1_5	พฤษภาคม	1
2_1	ธันวาคม	5
2_1	เมษายน	1
2_2	กันยายน	2
2_3	มกราคม	12
2_3	ธันวาคม	10
2_4	มกราคม	3

คำอธิบาย

- เลือกวัน/เดือน/ปีที่สั่งซื้อสินค้าจากตาราง Orders โดยแบ่งเป็นช่วงทั้งหมด 12 ช่วง (12เดือน) ซึ่งตั้งชื่อคอลัมน์ว่า month_th โดยจะเลือกคำสั่งซื้อที่อยู่ในช่วง 2024-01-01 จนถึง 2024-12-31
- จากนั้นทำการตรวจสอบว่าคำสั่งซื้อนั้น ๆ สั่งสินค้ารหัสใด (product_id) และจำนวนกี่ชิ้น (quantity) โดยใช้ JOIN order_details ON orders.order_id = order_details.order_id จะได้คำสั่งซื้อซึ่งมีรายการสั่งซื้อที่ประกอบด้วย รหัสสินค้า และจำนวนที่สั่งซื้อที่ตรงกับเดือนนั้น ๆ (month_th) จะแสดงเพียงแค่ รหัสสินค้า เดือน และจำนวนที่สั่งซื้อทั้งหมด โดยจะแบ่งตามเดือน และรหัสสินค้า จะได้ sum (quantity) group by month_th, product_id
- จากนั้นเรียงตามรหัสสินค้าและจำนวนที่ซื้อที่มากที่สุด order by product_id, sum (quantity) desc;

9. แสดงจำนวนคนในแต่ละเพศที่ซื้อสินค้าแต่ละรายการ

เหตุผล เพื่อวิเคราะห์ว่าสินค้าแต่ละรายการเป็นที่นิยมของเพศไหนมากกว่ากัน

SQL Query และผลลัพธ์

```
1 • select products.product_id,gender, count(*) as จำนวนผู้ใช้ from products
2 JOIN order_details od ON od.product_id = products.product_id
3 JOIN orders ON orders.order_id = od.order_id
4 JOIN users ON orders.username = users.username
5 JOIN Person_id on users.person_id_number = person_id.person_id_number
6 GROUP BY product_id , gender
7 ORDER BY product_id;
```

Result Grid Filter Rows:			
	product_id	gender	จำนวนผู้ใช้
▶	1_1	ชาย	2
	1_1	หญิง	3
	1_2	ชาย	2
	1_2	หญิง	1
	1_3	ชาย	2
	1_3	หญิง	5
	1_4	ชาย	3
	1_4	หญิง	3
	1_5	ชาย	3
	1_5	หญิง	2
	2_1	ชาย	2
	2_1	หญิง	1
	2_2	ชาย	1
	2_3	ชาย	1
	2_3	หญิง	1
	2_4	หญิง	3
	2_5	ชาย	1
	2_5	หญิง	2
	3_1	ชาย	1
	3_2	ชาย	2
	3_2	หญิง	5
	3_4	ชาย	1
	3_4	หญิง	3
	3_5	ชาย	1

คำอธิบาย

- ทำการเลือกรหัสสินค้า (product_id) จากตาราง Products เพื่อจะตรวจสอบว่าแต่ละสินค้ามีผู้ใช้ที่เป็นเพศไหนชื่อจำนวนกี่คน
- จากนั้น JOIN order_details เพื่อต้องการรู้คำสั่งซื้อที่ไหนบ้างที่ซื้อสินค้าแต่ละอัน เพราะในตาราง Orders ไม่สามารถรู้ถึงรหัสสินค้าได้ จึงต้องเชื่อมกับ Order_details จากนั้นเราสามารถเข้าถึง order_id ในตาราง Order_details ได้
- จากนั้นก็จะทำการเชื่อมตาราง Orders เพื่อเข้าถึง orders.username เพื่อบอกว่าผู้ใช้นั้นเป็นคนชื่อ
- จากนั้นก็ JOIN ตาราง Users เพื่อเข้าถึง users.Person_id_number เพื่อที่จะไป JOIN Person_id ON person_id.person_id_number = user.person_id_number เพื่อดึง gender ในตาราง Person_id
- จากนั้นพอรู้ว่าใครเป็นคนสั่งรายการสั่งซื้อไหนที่มีสินค้าอะไรบ้างก็จะทำการ COUNT(*) นับจำนวนแถว โดยแบ่งกลุ่มตามรหัสสินค้าและเพศ GROUP BY product_id, gender
- สุดท้ายเมื่อได้จำนวนผู้ใช้ของแต่ละเพศของแต่ละสินค้าแล้วก็แสดงผลโดยเรียงจากรหัสสินค้า ORDER BY product_id;

10. แสดงยอดคำสั่งซื้อรวมสินค้าทุกรายการ และจำนวนโปรโมชั่นทั้งหมดที่จัดในปี 2024 ของแต่ละร้านค้า เหตุผล ร้านค้าสามารถนำข้อมูลดังกล่าวมาวิเคราะห์ต่อได้ว่าการจัดโปรโมชันนั้นส่งผลต่อยอดขายหรือไม่ และคุ้มหรือไม่กับการจัดโปรโมชันให้กับลูกค้า

SQL Query และผลลัพธ์

```

1  SELECT sellers.seller_name AS store,
2      COUNT(DISTINCT orders.order_id) AS total_orders,
3      COUNT(DISTINCT promotions.promotion_id) AS total_promotions
4  FROM orders
5  JOIN order_details ON orders.order_id = order_details.order_id
6  AND orders.order_date BETWEEN '2024-01-01' AND '2024-12-31'
7  JOIN sellers ON SUBSTRING_INDEX(order_details.product_id, '_', -1) = sellers.seller_id
8  LEFT JOIN promotions ON sellers.seller_id = SUBSTRING_INDEX(promotions.product_id, '_', -1)
9  AND promotions.start_date BETWEEN '2024-01-01' AND '2024-12-31'
10 GROUP BY store
11 ORDER BY total_orders DESC;

```

store	total_orders	total_promotions
เจริญพาณิชย์	9	3
รุ่งเรืองพาณิชย์	8	2
สมพร	8	2
อุดมวิเศษ	7	4
โพธิ์ทอง	7	2

Result 12 x

Output

#	Time	Action	Message
145	17:34:50	SELECT sellers.seller_name AS store, COUNT(DISTINCT order...	5 row(s) returned
146	17:37:23	SELECT sellers.seller_name AS store, COUNT(DISTINCT order...	5 row(s) returned

คำอธิบาย

- ทำการเลือกชื่อผู้ขาย (seller_name) จากตารางผู้ขาย (Sellers) แล้วตั้งชื่อว่า store, เลือกหมายเลขคำสั่งซื้อสินค้า (order_id) จากตารางการสั่งซื้อสินค้า (Orders) แล้วตั้งชื่อว่า total_orders, เลือกรหัสโปรโมชั่น (promotion_id) จากตารางโปรโมชั่น (Promotion) แล้วตั้งชื่อว่า total_promotions
- JOIN ตาราง Order_details (ข้อมูลการสั่งซื้อ) โดยมีเงื่อนไข คือ หมายเลขคำสั่งซื้อสินค้า (order_id) ในตาราง Order_details และตารางคำสั่งซื้อ (Orders) ต้องเป็นรหัสเดียวกัน เพื่อจะตรวจสอบได้ว่าในคำสั่งซื้อดังกล่าวสั่งซื้อสินค้ารหัสใด โดยกำหนดให้ order_date ในตาราง Orders ต้องมีค่าระหว่าง 2024-01-01 ถึง 2024-12-31 เนื่องจากต้องการตรวจสอบแค่ข้อมูลในปี 2024
- JOIN ตาราง Sellers (ผู้ขาย) โดยมีเงื่อนไขคือเลขหลังสัญลักษณ์ '_' ของรหัสสินค้า (product_id) ใช้ SUBSTRING_INDEX (order_details.product_id, '-', -1) (ซึ่ง -1 คือเอาค่าหลังเครื่องหมาย '-') ในตาราง Order_details และรหัสผู้ขาย (seller_id) ในตารางผู้ขายต้องเป็นรหัสเดียวกัน เพื่อจะได้ตรวจสอบได้ว่าสินค้านี้เป็นสินค้าของผู้ขายใด
- JOIN ตาราง Promotions (โปรโมชั่น) โดยมีเงื่อนไข คือ รหัสผู้ขาย (seller_id) ในตารางผู้ขาย และเลขหลังสัญลักษณ์ '_' ของรหัสสินค้า (product_id) ใช้ SUBSTRING_INDEX (promotions.product_id, '-', -1) (ซึ่ง -1 คือเอาค่าหลังเครื่องหมาย '-') ในตาราง Promotions ต้องเป็นรหัสเดียวกัน เพื่อจะตรวจสอบได้ว่าโปรโมชั่นของสินค้านี้เป็นโปรโมชั่นของผู้ขายใด โดยกำหนดให้ start_date ในตาราง Promotions ต้องมีค่าระหว่าง 2024-01-01 ถึง 2024-12-31 เนื่องจากต้องการตรวจสอบแค่ข้อมูลในปี 2024
- จากนั้นพอรู้ว่าในแต่ละคำสั่งซื้อเป็นของผู้ขายใดก็จะทำการ COUNT (DISTINCT orders.order_id) นับจำนวนแถวของคำสั่งซื้อ และ COUNT (DISTINCT promotions.promotion_id) (DISTINCT เพื่อให้ค่าที่ซ้ำกันถูกนับแค่ 1 ครั้ง) นับจำนวนแถวของโปรโมชั่นของปี 2024 โดยแบ่งกลุ่มตามชื่อผู้ขาย GROUP BY store